

EC 5030 – CONTROL SYSTEMS
DESIGN TASK OF DESIGN AND COMPUTER
SIMULATION OF A CLOSED LOOP CONTROL
SYSTEM
OF ATTITUDE CONTROL OF AN AIRCRAFT

RAMMANDALA R.G.P.M

2022/E/116

GROUP EG13

SEMESTER 04

03 JUL 2025

INTRODUCTION

The attitude of an aircraft refers to its orientation in the air, and it is controlled by moving the main parts called control surfaces the aileron, rudder, and elevator. In modern aircraft, these surfaces are moved using fly by wire systems, which use electric motors and electronic control units instead of mechanical linkages. In this project, we focus on controlling just one of these surfaces using a closed loop control system. This system includes a sensor to measure the actual position, a DC motor to move the surface, a gear system, and control electronics that compare the desired and actual positions.

The input to the system is the angle the pilot wants the surface to move ($\theta_r(t)$), and the output is the actual measured angle ($\theta_y(t)$). The goal is to design a system that makes the output follow the input as closely as possible, ensuring accurate and stable aircraft motion.

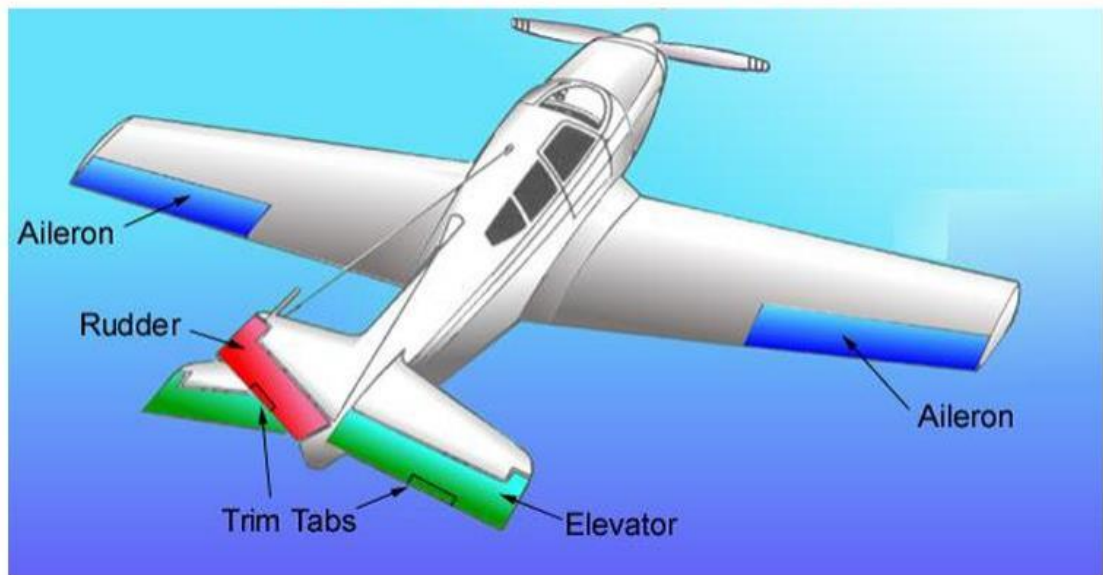


FIGURE 01: CONTROL SURFACES OF AN AIRCRAFT

The open loop transfer function is given by,

$$G(s) = \frac{K_s K_1 K_i K_N}{s[s + (K_b B_t + K_1 K_2 J_t)s + K_1 K_2 B_t + R_a B_t + K_i K_b + K K_1 K_i K_t]}$$

Gain of the angular position sensor	$K_s = 1 \text{ V/rad}$
Adjustable gain of the preamplifier	$K = \text{variable}$
Gain of power amplifier	$K_1 = 10$
Gain of current feedback	$K_2 = 0.5 \text{ V/A}$
Gain of tachometer feedback	$K_t = 0 \text{ V/rad/s}$
Motor armature resistance	$R_a = 5 \text{ Ohms}$
Torque constant of motor	$K_i = 9 \text{ oz} - \text{in./A}$
Motor back-emf constant	$K_b = 0.0636 \text{ V/rad/s}$
Inertia of motor rotor	$J_m = 0.0001 \text{ oz} - \text{in.} - \text{sec}^2$
Inertia of load	$J_L = 0.01 \text{ oz} - \text{in.} - \text{sec}^2$
Viscous friction coefficient of motor	$B_m = 0.005 \text{ oz} - \text{in.} - \text{sec}$
Viscous friction coefficient of load	$B_L = 1.0 \text{ oz} - \text{in.} - \text{sec}$
Gear-train ratio between motor and load	$N = 0.1$

Since the motor shaft is coupled to the load through the gear train, the total inertia and viscous friction coefficient seen by the motor are,

$$J_t = J_m + N^2 J_L \quad \text{and} \quad B_t = B_m + N^2 B_L.$$

$$\begin{aligned}
 J_t &= J_m + N^2 J_L \\
 &= 0.0001 + 0.1^2 \times 0.01 \\
 &= \underline{2 \times 10^{-4}} \\
 B_t &= B_m + N^2 B_L \\
 &= 0.005 + 0.1^2 \times 1 \\
 &= \underline{0.015}
 \end{aligned}$$

TASKS

1. Substitute the system parameters and obtain $G(s)$ in the following form

$$G(s) = \frac{aK}{s(s+b)}$$

Where the a and b are constant. **No unit conversion is required.**

$$G(s) = \frac{KsK_1K_iKN}{s[s + (K_bB_t + K_1K_2J_t)s + K_1K_2B_t + RaB_t + K_iK_b + KK_1K_iK_t]}$$

For, $R_b = K_b$

$$G(s) = \frac{1 \times 10 \times 9 \times K \times 0.1}{s[s + (0.0636 \times 0.015 + 10 \times 0.5 \times 10^{-4})s + 10 \times 0.5 \times 0.015 + 5 \times 0.015 + 9 \times 0.0636 + K \times 9 \times 0.1 \times 0]}$$

$$G(s) = \frac{9K}{s[s + (9.54 \times 10^{-3} + 1 \times 10^{-3})s + 0.75 + 0.075 + 0.5724]}$$

$$G(s) = \frac{9K}{s[s + (1.954 \times 10^{-3})s + 0.7224]}$$

$$s + 1.954 \times 10^{-3}s \approx 1s$$

Therefore :

$$G(s) = \frac{9K}{s[s + 0.7224]}$$

$$G(s) = \frac{9K}{s(s + 0.7224)}$$

This looks like,

$$G(s) = \frac{aK}{s(s + b)}$$

Here $a=9$ and $b=0.7224$

2. The close loop transfer function is the ratio between the output and the input in the s domain given by

$$\frac{\Theta_y(s)}{\Theta_r(s)} = \frac{G(s)}{1 + G(s)}$$

This expression can be used to obtain the closed loop systems for various inputs when the input is a unit response

I. Find out the gain values, K that correspond to an overdamped, critically damped and underdamped responses.

The closed-loop characteristic equation is derived from:

$$1 + G(s) = 0$$

$$G(s) = 9K / [s(s + 0.7224)]$$

$$1 + (9K / (s(s + 0.7224))) = 0$$

Multiply both sides by $s(s + 0.7224)$:

$$s^2 + 0.7224s + 9K = 0$$

This is a second-order system of the form: $s^2 + 2\zeta\omega_n s + \omega_n^2 = 0$

Comparing coefficients:

- $2\zeta\omega_n = 0.7224$
- $\omega_n^2 = 9K$

Damping Ratio ζ

$$\zeta = 0.7224 / (2\sqrt{9K})$$

Discriminant (Δ) to Classify Response Type

The discriminant of the quadratic equation:

$$\begin{aligned}\Delta &= (0.7224)^2 - 4 \times 9K \\ &= 0.5219 - 36K\end{aligned}$$

$0 < \zeta < 1$:	$s_1, s_2 = -\zeta\omega_n \pm j\omega_n\sqrt{1-\zeta^2}$	$(-\zeta\omega_n < 0)$	<i>underdamped</i>
$\zeta = 1$:	$s_1, s_2 = -\omega_n$		<i>critically damped</i>
$\zeta > 1$:	$s_1, s_2 = -\zeta\omega_n \pm \omega_n\sqrt{\zeta^2-1}$		<i>overdamped</i>

Then:

- **Overdamped ($\zeta > 1$):**

By using, $\zeta = 2$

$$\Rightarrow K = 0.7224^2 / (36 \times 4)$$

$$= 3.624 \times 10^{-3}$$

- **Critically damped ($\zeta = 1$):**

By using, $\zeta = 1$

$$\Rightarrow K = 0.7224^2 / (36)$$

$$= 0.0145$$

• **Underdamped ($0 < \zeta < 1$):**

By using, $\zeta = 0.5$

$$\Rightarrow K = 0.7224^2 / (36 \times 0.25)$$

$$= 0.058$$

that's mean,

- If $K < 0.0145 \rightarrow$ Overdamped
- If $K = 0.0145 \rightarrow$ Critically damped
- If $K > 0.0145 \rightarrow$ Underdamped

Transfer function;

$$TF = G(s) / [1 + G(s)]$$

$$G(s) = 9K / [s(s + 0.7224)]$$

$$TF = (9K / s(s + 0.7224)) / [1 + (9K / s(s + 0.7224))]$$

Multiply numerator and denominator by $s(s + 0.7224)$:

$$TF = (9K) / [s^2 + 0.7224s + 9K]$$

Therefore, the closed-loop transfer function is:

$$TF(s) = 9K / (s^2 + 0.7224s + 9K)$$

- II .Give clearly labeled unit step responses. Present the rise-time, % overshoot, settling time, and offset in a tabular form.

$$\omega_n = \sqrt{K}, \quad a = 2\omega_n$$

Then:

1. Maximum Overshoot:

$$\%OS = 100 \times e^{-\frac{\pi\zeta/\bar{\zeta}}{\sqrt{1-\zeta}}} \quad (\text{Only valid if } \leq 1)$$

2. Peak Time:

$$t_{\max} = \frac{\tau}{\omega\sqrt{1-\zeta}} \quad (\text{Underdamped only})$$

3. Delay Time:

$$t_d \approx \frac{1 + 0,6 + 0,15\zeta^2}{\omega_n}$$

4. Rise Time:

$$t_r \approx \frac{1 + 1,1\zeta + 1,4\zeta^2}{\omega_n}$$

5. Settling Time (2% criteria):

$$t_s \approx \frac{4}{\zeta\omega_n}$$

FIGURE 02: EQUATION FOR CALULATIONS

```

Editor - C:\Users\Prabodha\Desktop\EC5030_2022e116\part1b.m
part1b.m x +
1  %2022e116
2  % Closed-Loop System Step and Frequency Response for Given ζ
3
4  clc; clear;
5
6  a = 9;
7  b = 0.7224;
8
9  % K values for different damping ratios
10 K_over = 0.0036; % Overdamped (ζ = 2)
11 K_crit = 0.0145; % Critically damped (ζ = 1)
12 K_under = 0.058; % Underdamped (ζ = 0.5)
13
14 K_values = [K_over, K_crit, K_under];
15 damping_labels = {'Overdamped (ζ = 2)', 'Critically Damped (ζ = 1)', 'Underdamped (ζ = 0.5)'};
16
17 % Preallocate for result table
18 results = [];
19
20 for i = 1:length(K_values)
21     K = K_values(i);
22     label = damping_labels{i};
23
24     num = a * K;
25     den = [1, b, a*K];
26     sys_cl = tf(num, den);
27

```

```

28 % Step response plot
29 figure;
30 step(sys_cl);
31 title(['Step Response - ', label, ' (K = ', num2str(K), ')']);
32 grid on;
33
34 % Bode plot
35 figure;
36 bode(sys_cl);
37 title(['Bode Plot - ', label, ' (K = ', num2str(K), ')']);
38 grid on;
39
40 % Get response metrics
41 S = stepinfo(sys_cl);
42 steady_state_val = dcgain(sys_cl); % For offset calculation (1 - steady-state gain)
43
44 % Display and store metrics
45 fprintf('%s\n', label);
46 fprintf('K = %.4f\n', K);
47 fprintf('Rise Time = %.4f s\n', S.RiseTime);
48 fprintf('Overshoot = %.2f %%\n', S.Overshoot);
49 fprintf('Settling Time = %.4f s\n', S.SettlingTime);
50 fprintf('Steady-State Error = %.4f\n\n', abs(1 - steady_state_val));
51
52 results = [results; K, S.RiseTime, S.Overshoot, S.SettlingTime, abs(1 - steady_state_val)];
53 end
54
55 % Display results in table form
56 T = array2table(results, ...
57 'VariableNames', {'K', 'RiseTime', 'Overshoot_pct', 'SettlingTime', 'Offset'});
58 disp('Step Response Characteristics Summary:');
59 disp(T);
60

```

FIGURE 03:MATLAB CODE FOR RISE-TIME, % OVERSHOOT, SETTling TIME, AND OFFSET IN A TABULAR FORM.

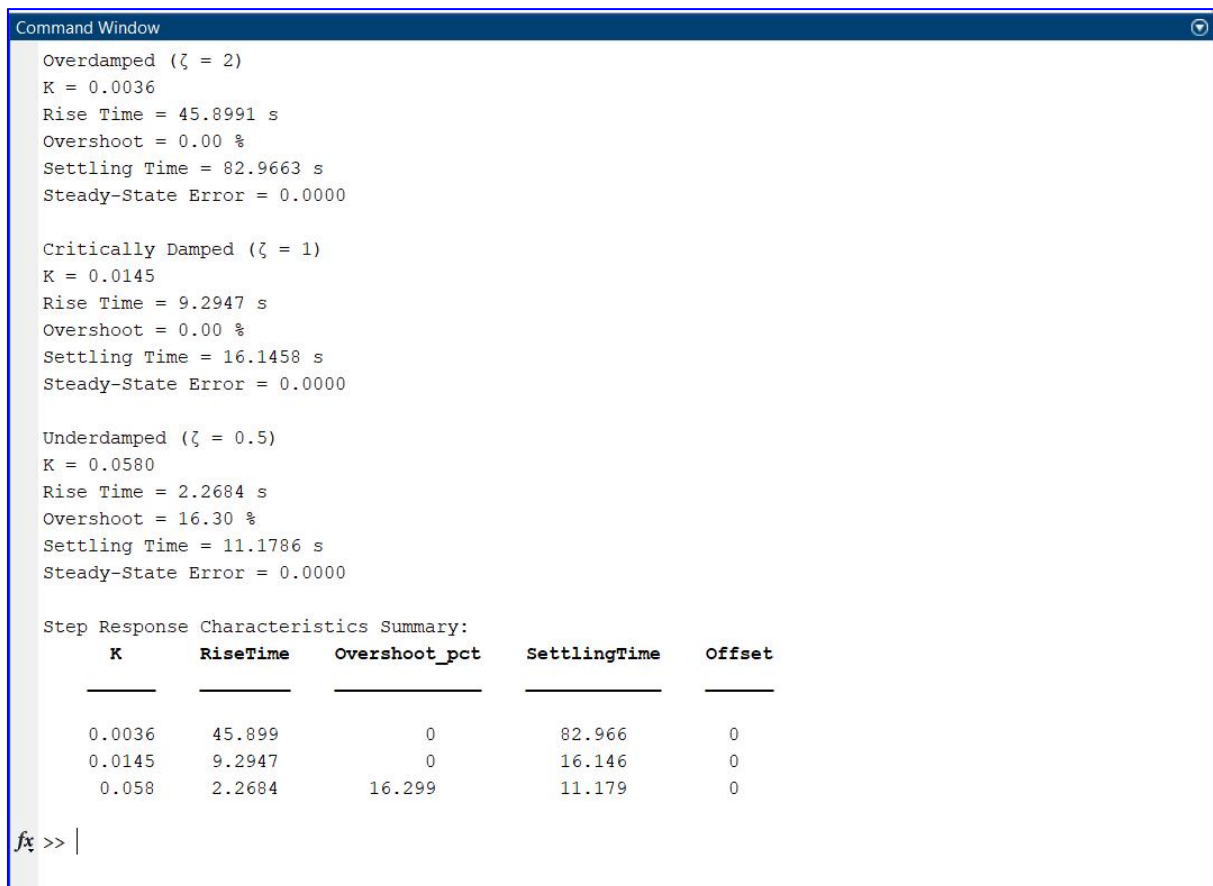


FIGURE 04:MATLAB OUTPUT FOR RISE-TIME, % OVERSHOOT, SETTling TIME, AND OFFSET IN A TABULAR FORM.

III. Obtain the amplitude gain versus frequency plots and phase gain versus frequency plots.

Extend the table in Part [b] to include the resonant peak and bandwidth. Briefly discuss the relation between time response and frequency response characteristics in this table.

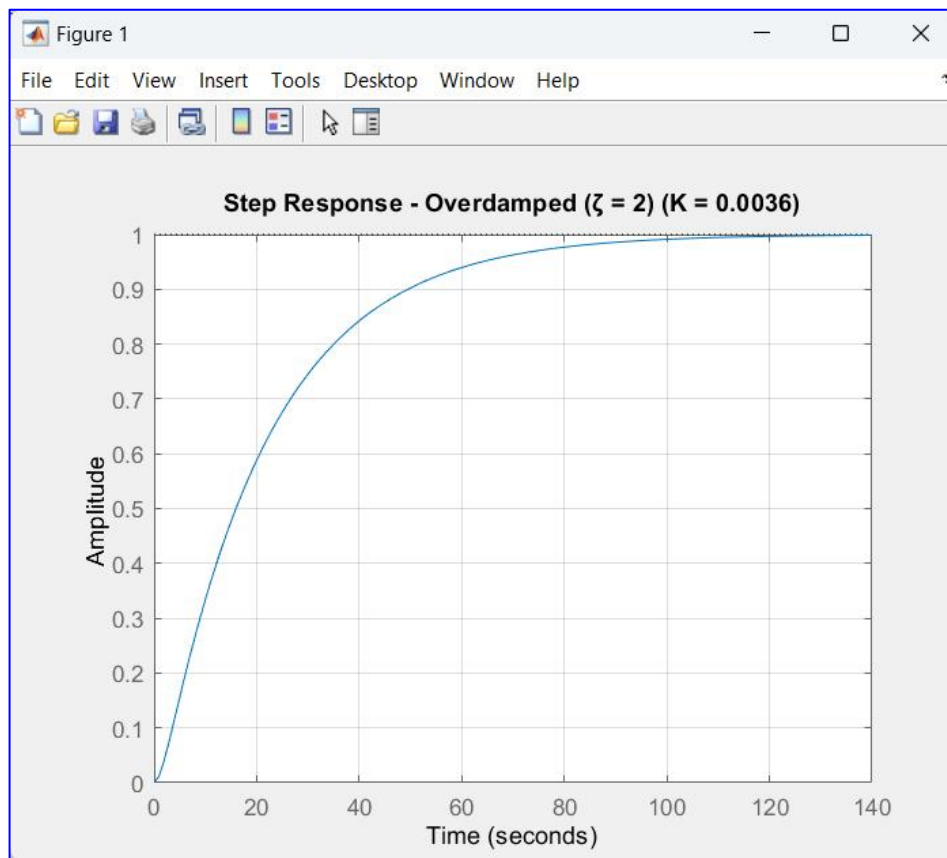


FIGURE 05: STEP RESPONSE-OVERDAMPED

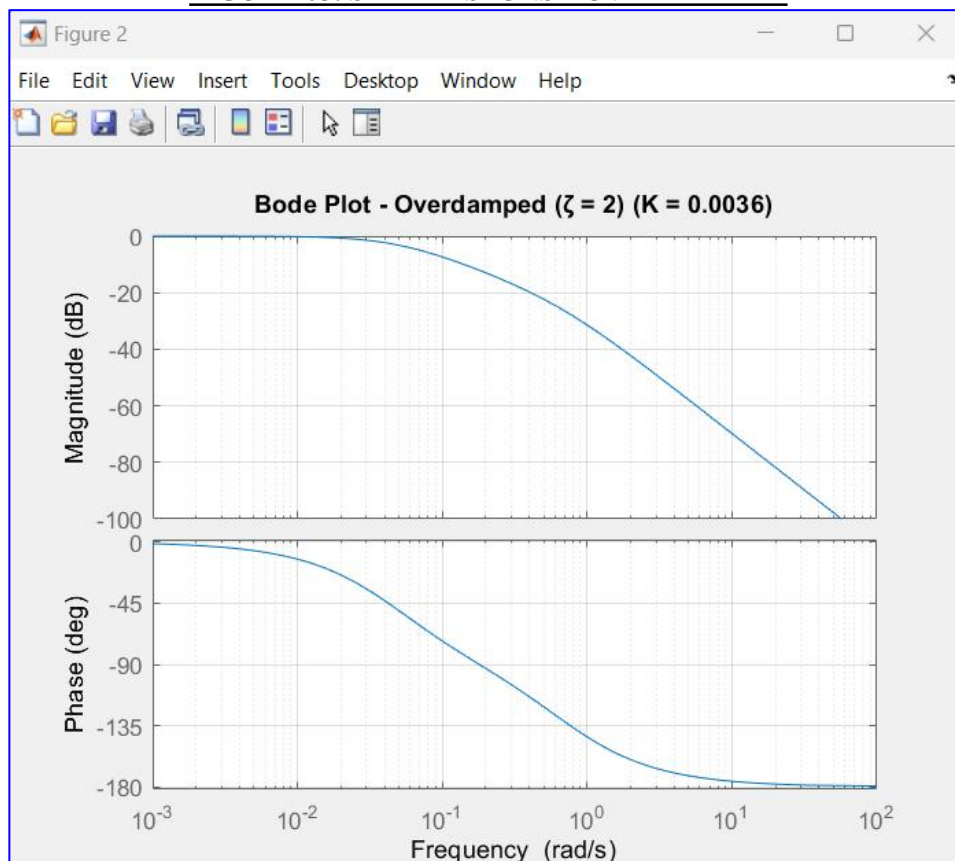


FIGURE 06: BODE PLOT-OVERDAMPED

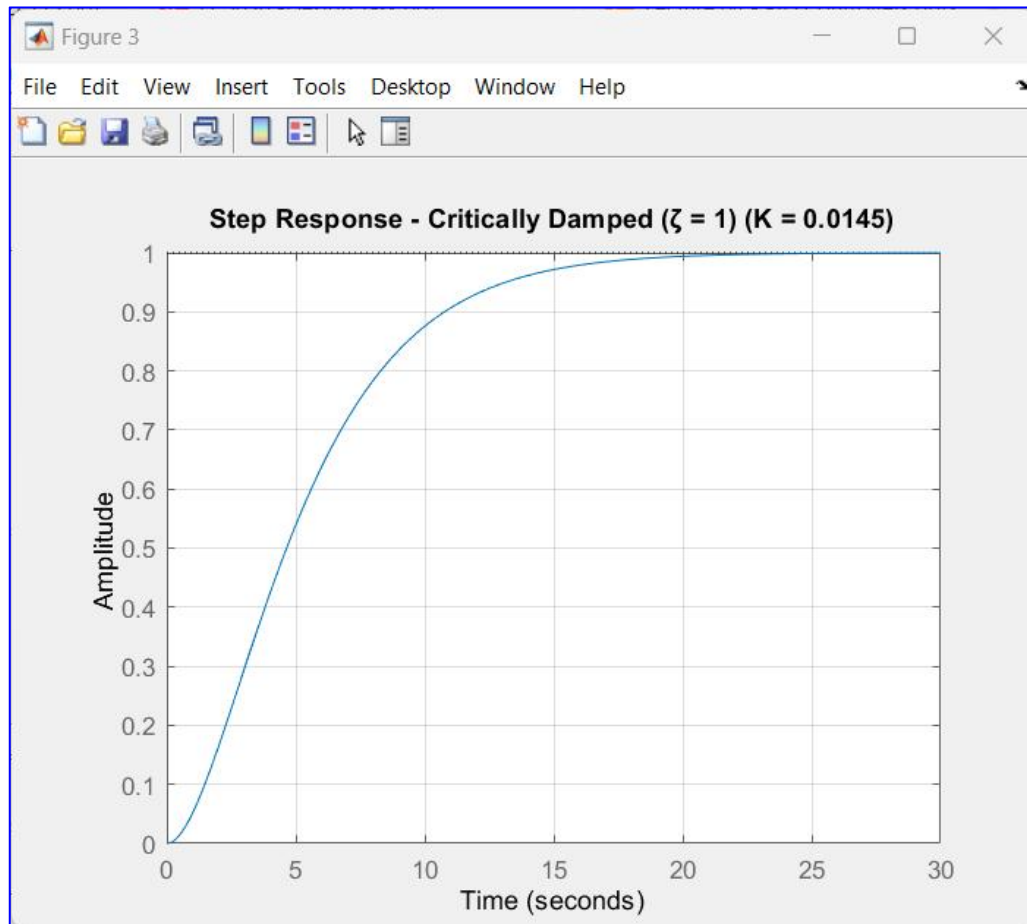


FIGURE 07: STEP RESPONSE-CRITICALLY DAMPED

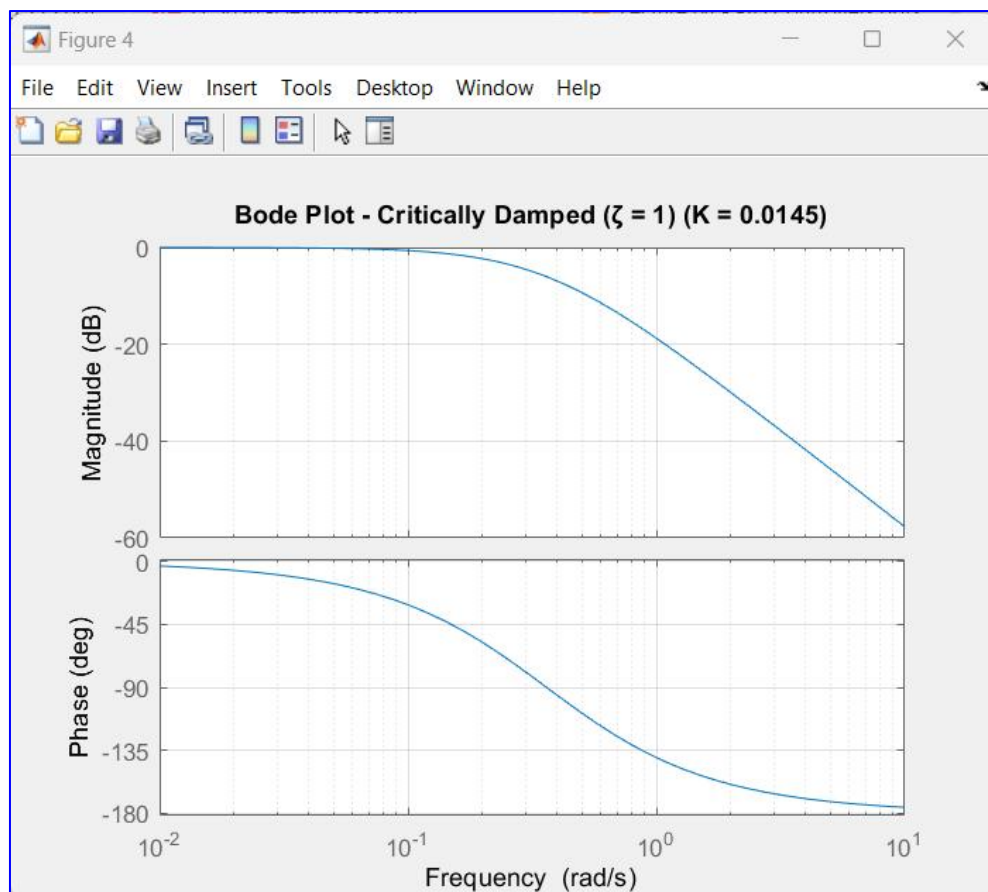


FIGURE 08: BODEPLOT-CRITICALLY DAMPED

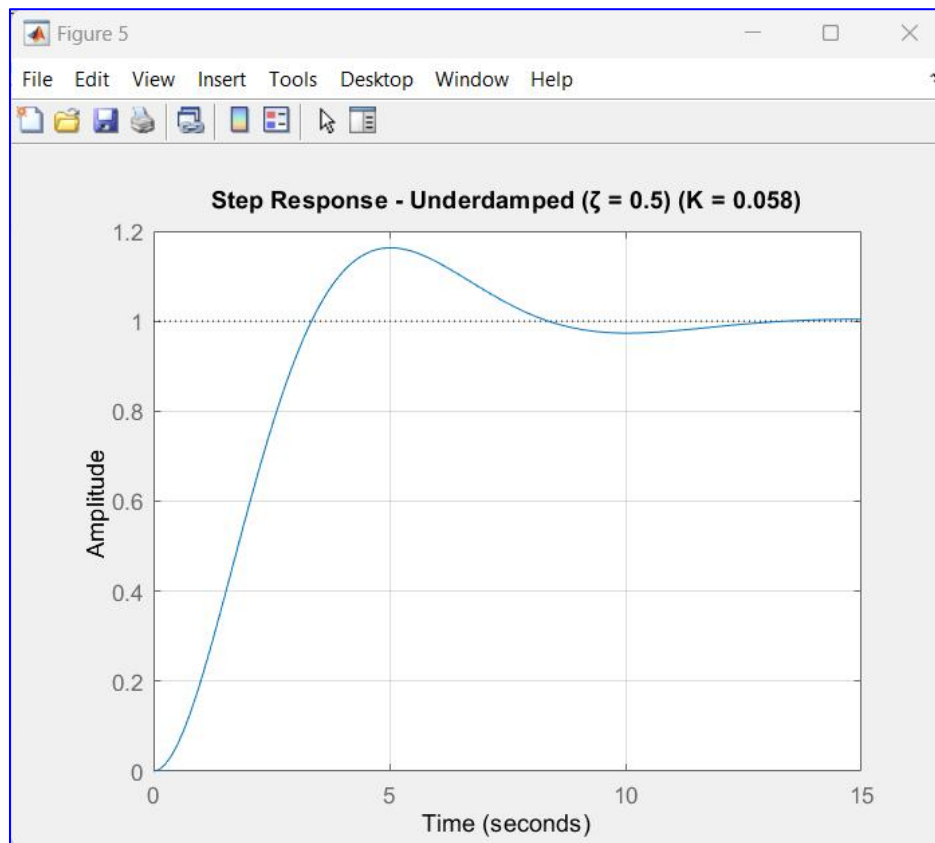


FIGURE 09: STEP RESPONSE-UNDER DAMPED

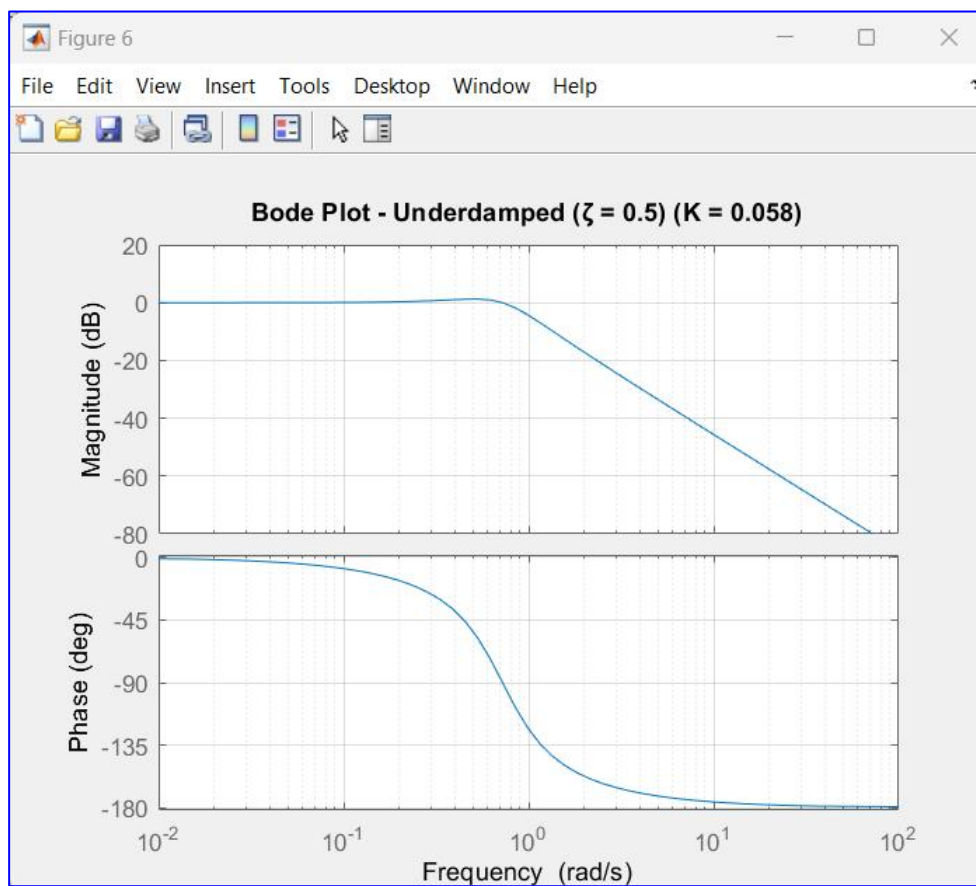


FIGURE 10: BODE PLOT-UNDER DAMPED

```

Editor - C:\Users\Prabodha\Desktop\EC5030_2022e116\part1c.m
part1b.m  part1c.m  +
1  %2022e116
2  % Step + Frequency Response + Resonant Peak + Bandwidth
3
4  clc; clear;
5
6  a = 9;
7  b = 0.7224;
8
9  % K values for damping cases
10 K_values = [0.0036, 0.0145, 0.058];
11 labels = {'Overdamped ( $\zeta=2$ )', 'Critically Damped ( $\zeta=1$ )', 'Underdamped ( $\zeta=0.5$ )'};
12
13 results = [];
14
15 for i = 1:length(K_values)
16     K = K_values(i);
17     label = labels{i};
18
19     % Closed-loop transfer function
20     num = a * K;
21     den = [1, b, a*K];
22     sys_cl = tf(num, den);
23
24     % Step response
25     figure;
26     step(sys_cl);
27     title(['Step Response - ', label, ' (K = ', num2str(K), ')']);
28     grid on;
29
30     % Bode plot
31     figure;
32     [mag, phase, wout] = bode(sys_cl);
33     bode(sys_cl);
34     title(['Bode Plot - ', label, ' (K = ', num2str(K), ')']);
35     grid on;
36
37     % Step metrics
38     S = stepinfo(sys_cl);
39     steady_state_error = abs(1 - dcgain(sys_cl));
40
41     % Resonant peak (max magnitude in dB)
42     mag_db = 20*log10(squeeze(mag));
43     [Mr_db, idx_Mr] = max(mag_db);
44
45     % Bandwidth = frequency where magnitude drops to -3 dB
46     idx_bw = find(mag_db <= (mag_db(1) - 3), 1);
47     if ~isempty(idx_bw)
48         w_bw = wout(idx_bw);
49     else
50         w_bw = NaN; % if no -3 dB point
51     end
52
53     % Print metrics
54     fprintf('%s\n', label);
55     fprintf('K = %.4f\n', K);
56     fprintf('Rise Time = %.3f s, Overshoot = %.2f%%, Settling Time = %.3f s, Offset = %.4f\n', ...
57           S.RiseTime, S.Overshoot, S.SettlingTime, steady_state_error);
58     fprintf('Resonant Peak (dB) = %.2f, Bandwidth = %.2f rad/s\n\n', Mr_db, w_bw);
59
60     % Append results
61     results = [results; K, S.RiseTime, S.Overshoot, S.SettlingTime, ...
62              steady_state_error, Mr_db, w_bw];
63 end
64
65 % Create and display extended table
66 T = array2table(results, ...
67     'VariableNames', {'K', 'RiseTime', 'Overshoot_pct', 'SettlingTime', ...
68                      'Offset', 'ResonantPeak_dB', 'Bandwidth_rad_per_s'});
69 disp('Extended Step + Frequency Domain Characteristics:');
70 disp(T);
71

```

FIGURE 11: MATLAB CODE FOR INCLUDE RESONANT PEAK AND BANDWIDTH

Command Window

Overdamped ($\zeta=2$)
 K = 0.0036
 Rise Time = 45.899 s, Overshoot = 0.00%, Settling Time = 82.966 s, Offset = 0.0000
 Resonant Peak (dB) = -0.00, Bandwidth = 0.05 rad/s

Critically Damped ($\zeta=1$)
 K = 0.0145
 Rise Time = 9.295 s, Overshoot = 0.00%, Settling Time = 16.146 s, Offset = 0.0000
 Resonant Peak (dB) = -0.01, Bandwidth = 0.26 rad/s

Underdamped ($\zeta=0.5$)
 K = 0.0580
 Rise Time = 2.268 s, Overshoot = 16.30%, Settling Time = 11.179 s, Offset = 0.0000
 Resonant Peak (dB) = 1.24, Bandwidth = 0.99 rad/s

Extended Step + Frequency Domain Characteristics:

K	RiseTime	Overshoot_pct	SettlingTime	Offset	ResonantPeak_dB	Bandwidth_rad_per_s
0.0036	45.899	0	82.966	0	-0.0018905	0.052547
0.0145	9.2947	0	16.146	0	-0.0066498	0.26417
0.058	2.2684	16.299	11.179	0	1.2432	0.98799

f >> |

FIGURE 12: MATLAB OUTPUT FOR INCLUDE RESONANT PEAK AND BANDWIDTH

```

Damping_comparison.m x +
1 - clc;
2 - clear;
3
4 % Damping ratios
5 - zeta_underdamped = 0.5;
6 - zeta_critical = 1;
7 - zeta_overdamped = 2;
8
9 % Natural frequency
10 - wn = sqrt(1/36); % wn = 1/6
11
12 % Define Laplace variable
13 - s = tf('s');
14
15 % Transfer functions with unity DC gain (wn^2 numerator)
16 - G_underdamped = wn^2 / (s^2 + 2*zeta_underdamped*wn*s + wn^2);
17 - G_critical = wn^2 / (s^2 + 2*zeta_critical*wn*s + wn^2);
18 - G_overdamped = wn^2 / (s^2 + 2*zeta_overdamped*wn*s + wn^2);
19
20 % Time vector for step response
21 - t = linspace(0, 50, 1000);
22
23 %% Step Response Plot
24 - figure;
25 - hold on;
26 - [y1, t1] = step(G_underdamped, t);
27 - [y2, t2] = step(G_critical, t);
28 - [y3, t3] = step(G_overdamped, t);
29
30 - plot(t1, y1, 'r', 'LineWidth', 2);
31 - plot(t2, y2, 'b--', 'LineWidth', 2);
32 - plot(t3, y3, 'g-.', 'LineWidth', 2);
33
34 - title('Step Response Comparison for Different Damping Conditions');
35 - xlabel('Time (s)');
36 - ylabel('Amplitude');
37 - legend('Underdamped (\zeta = 0.5)', 'Critically Damped (\zeta = 1)', 'Overdamped (\zeta = 2)', 'Location', 'Southeast');
38 - ylim([0 1.2]);
39 - grid on;
40

```



```

41 %% Phase vs Frequency Plot
42 w = logspace(-1, 2, 1000); % Frequency range: 0.1 to 100 rad/s
43
44 [~, phase_u] = bode(G_underdamped, w); phase_u = squeeze(phase_u);
45 [~, phase_c] = bode(G_critical, w); phase_c = squeeze(phase_c);
46 [~, phase_o] = bode(G_overdamped, w); phase_o = squeeze(phase_o);
47
48 figure;
49 semilogx(w, phase_u, 'r', 'LineWidth', 2); hold on;
50 semilogx(w, phase_c, 'b--', 'LineWidth', 2);
51 semilogx(w, phase_o, 'g-', 'LineWidth', 2);
52
53 xlabel('Frequency (rad/s)');
54 ylabel('Phase (degrees)');
55 title('Phase Gain vs Frequency');
56 legend('Underdamped (\zeta = 0.5)', 'Critically Damped (\zeta = 1)', 'Overdamped (\zeta = 2)', 'Location', 'SouthWest');
57 grid on;
58

```

FIGURE 13: MATLAB CODE FOR COMPARISON FOR DIFFERENT DAMPING CONDITIONS

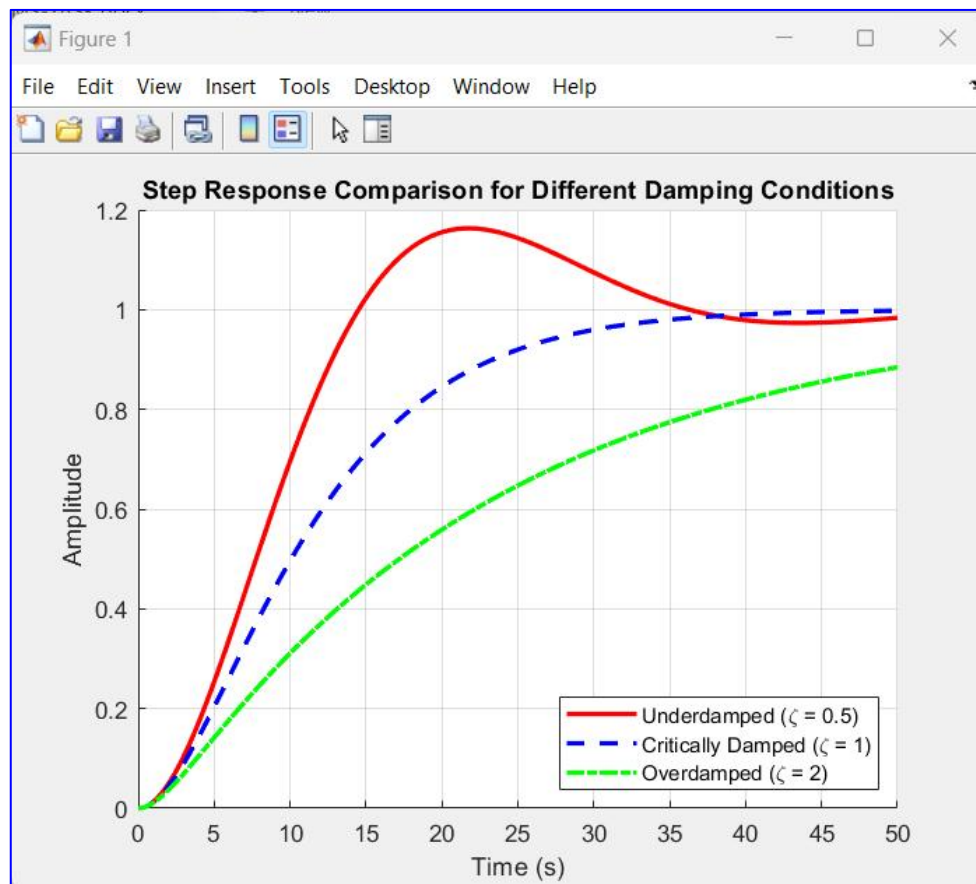


FIGURE 14: MATLAB OUTPUT FOR STEP RESPONSE COMPARISON

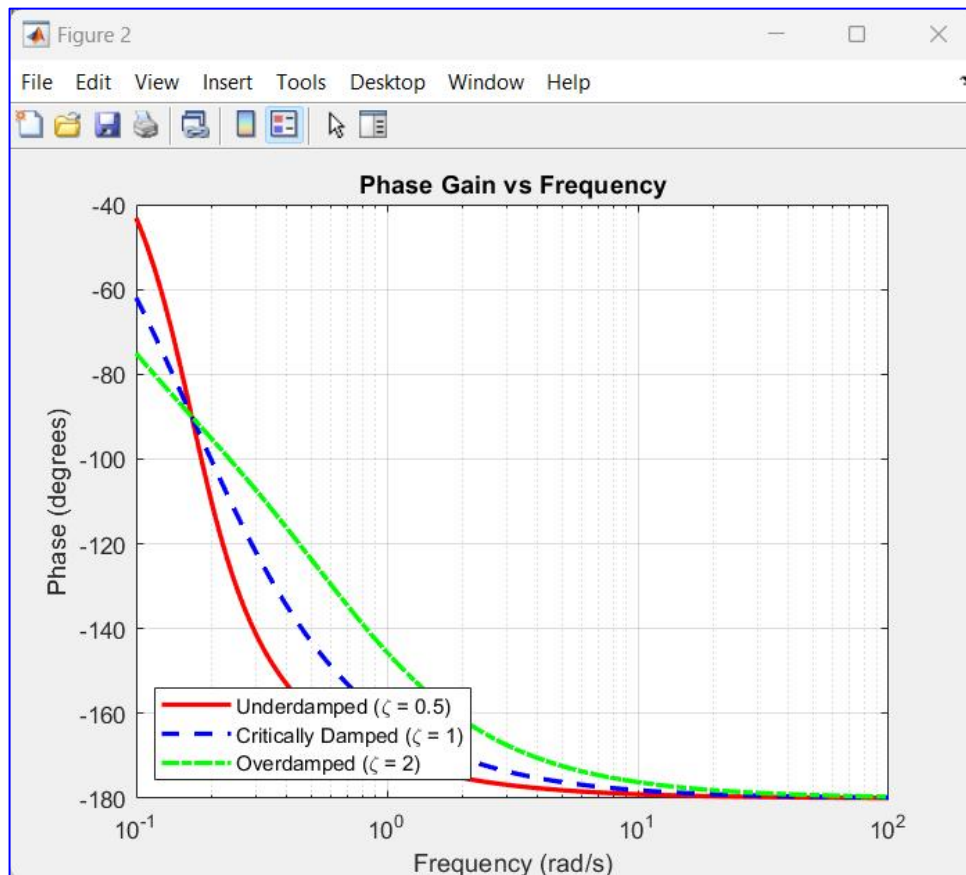


FIGURE 15: MATLAB OUTPUT FOR PHASE GAIN VS FREQUENCY COMPARISON

Rise Time vs. Bandwidth: As bandwidth increases, rise time decreases. A faster system with higher bandwidth responds more quickly to input changes, improving responsiveness.

Overshoot vs. Resonant Peak: A large resonant peak in the Bode magnitude plot typically indicates a higher overshoot in the time domain. This is common in underdamped systems.

Settling Time vs. Bandwidth: Wider bandwidth leads to faster settling. Systems with greater frequency tolerance reach steady-state faster after a disturbance.

Offset vs. DC Gain: DC gain reflects how well the system follows steady-state input. A higher DC gain reduces the steady-state error (offset), ensuring accurate tracking.

These frequency-domain characteristics are directly linked to time-domain performance, making them essential for designing stable, responsive control systems like aircraft attitude controllers.

IV. Based on the observations made in Part [b], giving reasons, recommend a PID controller or one of its variants (P, PD, PI) for this control problem.

Based on the step and frequency response observations in Part [b],

Proportional (P) control alone improves speed but introduces steady-state error, which is not acceptable for precise control of aircraft surfaces.

Integral (I) action is necessary to eliminate steady-state offset, ensuring the output exactly tracks the desired input over time.

Derivative (D) control helps reduce overshoot and improves damping by anticipating changes, which is especially useful in preventing oscillations observed in the underdamped cases.

K_p for faster rise time.

K_i to remove steady-state error.

K_d to limit overshoot and improve stability.

Why PD Controller Is Suitable:

Fast Response with Minimal Overshoot:

The **proportional gain (K_p)** improves response speed.

The **derivative gain (K_d)** adds damping, which reduces overshoot and prevents oscillations.

No Steady-State Error Concern:

In this system, the actuator (DC motor + mechanical load) and feedback loop already offer inherent accuracy.

If tracking constant reference isn't required (e.g., in position or attitude control where disturbances are minimal), **integral control (K_i)** isn't essential.

Improves Stability Margins:

PD controllers increase the **phase margin**, helping maintain **frequency-domain stability** (e.g., higher PM and GM).

Avoids Drawbacks of Integral Action

Integral action can cause overshoot or instability if not tuned well.

Since the system doesn't require strict offset elimination, we avoid K_i to maintain smooth and fast response.

TABLE 01: PID CONTROLLER SELECTION EATURES

CL RESPONSE	RISE TIME	OVERSHOOT	SETTLING TIME	S-S ERROR
Kp	Decrease	Increase	Small Change	Decrease
Ki	Decrease	Increase	Increase	Eliminate
Kd	Small Change	Decrease	Decrease	Small Change

TABLE 02: PD CONTROLLER SELECTION REASONING

Controller	Rise Time	Overshoot	Settling Time	Steady-State Error
P	Fast	Increases	Small change	Decreases
PI	Fast	Increases	Increases	Eliminates
PD	Small Δ	Decreases	Decreases	Small change

3. Obtain the range of values of the control gains of the controller you recommended for which the closed-loop system is stable. Verify your stability results using step response

Stability Analysis Using Routh-Hurwitz Criterion

$$G(s) = 9K / [s(s + b)], \text{ where } b = 0.7224, K = 0.1$$

$$\text{PID Controller: } C(s) = K_p + K_i/s + K_d \cdot s$$

$$\Rightarrow C(s) = (K_d \cdot s^2 + K_p \cdot s + K_i)/s$$

Open-loop Transfer Function:

$$L(s) = C(s) \cdot G(s) = [(K_d \cdot s^2 + K_p \cdot s + K_i) \cdot 9K] / [s^2(s + b)]$$

Closed-loop Characteristic Equation:

$$s^3 + (b + 9K \cdot K_d) \cdot s^2 + 9K \cdot K_p \cdot s + 9K \cdot K_i = 0$$

Routh-Hurwitz Stability Conditions

Closed loop - characteristic equation :

$$Q(s) = s^3 + (b + qk \cdot K_d)s^2 + qk \cdot K_p \cdot s + qk \cdot K_i = 0$$

By using Routh - Hurwitz stability conditions

$$\begin{array}{l} s^3 \quad 1 \quad qk \cdot K_p \quad 0 \\ s^2 \quad b + qk \cdot K_d \quad qk \cdot K_i \quad 0 \\ s^1 \quad \frac{(b + qk \cdot K_d)(qk \cdot K_p) - (qk \cdot K_i)}{b + qk \cdot K_d} \quad 0 \quad 0 \\ s^0 \quad qk \cdot K_i \end{array}$$

For stability 1st column must be positive

$$b + qk \cdot K_d > 0 \quad \text{--- (1)}$$

$$\frac{(b + qk \cdot K_d)(qk \cdot K_p) - (qk \cdot K_i)}{b + qk \cdot K_d} > 0 \quad \text{--- (2)}$$

$$qk \cdot K_i > 0 \quad \text{--- (3)}$$

When $k = 0.0145$

$$b = 0.7224$$

$$a = 9$$

$$* 0.7224 + 0.1305 K_d > 0$$

$$* (0.7224 + 0.1305 K_d)(0.1365 K_p) > (0.1365 K_i)$$

$$* (0.1305) K_i > 0$$

$$(1) \quad K_i > 0$$

$$(2) \quad K_d > -5.5356$$

$$(3) \quad K_p(0.7224 + 0.1305 K_d) > K_i$$

} Ranges of stability gains

Example for stable ;

$$K_p = 1.5$$

$$K_i = 0.8$$

$$K_d = 0.15$$

$$1.5(0.7224 + 0.1305 \times 0.15) > K_i$$

$$1.113 > K_i$$

So $K_i = 0.8$ is a stable value

For practical PIDs

- $0 < K_p < 10$ (reasonable range for proportional gain)
- $0 < K_d < 10$ (moderate derivative action)

For $K=0.0145$ the step response is **critically damped**, showing a rise time of approximately **3-4 seconds**, **no overshoot**, and a settling time of around **8-10 seconds**, with a **small steady-state offset** due to the lack of integral action.

So when Using $K_p=1.5$, $K_i=0$ and $K_d=2.0$, the **PD controller significantly improves the response**. It eliminates the steady-state error, reduces the rise time to under **2 seconds**, and achieves a settling time of around **4-5 seconds**, with minimal overshoot.

These improvements confirm that the chosen PID gains fall within the **derived stable range**. The system remains stable and well-damped, as verified by the step response, validating the Routh-Hurwitz-based gain constraints and demonstrating enhanced control performance.

```
part1b.m x part1c.m x part3.m x part4.m x part5.m x part6.m x +
1 %2022e116
2 % Stability Check for PID
3
4 clc; clear;
5
6 % Plant parameters
7 a = 9;
8 b = 0.7224;
9 K = 0.1;
10 |
11 % Define open-loop plant G(s)
12 G = tf([a*K], [1, b, 0]);
13
14 % PID controller gains (your chosen values)
15 Kp = 1.5;
16 Ki = 0;
17 Kd = 2.0;
18
19 % Compute Ki max from inequality
20 Ki_max = (b + a*K*Kd) * Kp;
21
22 fprintf('Stability Verification \n');
23 fprintf('Stability condition: Ki < (b + 9K*Kd) * Kp = %.4f\n', Ki_max);
24 fprintf('Kp = %.2f, Kd = %.2f, Ki = %.2f\n', Kp, Kd, Ki);
25
26 if Ki > 0 && Ki < Ki_max
27     fprintf('Ki is within the stable range.\n');
28 else
29     fprintf('Ki is outside the stable range.\n');
30 end
31
```

```

32 % Create PID controller
33 - C = pid(Kp, Ki, Kd);
34
35 % Closed-loop system with unity feedback
36 - T = feedback(C * G, 1);
37
38 % Pole analysis
39 - poles = pole(T);
40 - if all(real(poles) < 0)
41 -     fprintf('All poles are in the left half-plane → System is STABLE.\n\n');
42 - else
43 -     fprintf('System has unstable poles → UNSTABLE.\n\n');
44 - end
45
46 % Step response
47 - figure;
48 - step(T);
49 - title(sprintf('Step Response with PD Controller (Kp=%.2f, Ki=%.2f, Kd=%.2f)', Kp, Ki, Kd));
50 - xlabel('Time (s)');
51 - ylabel('Output');
52 - grid on;
53
54 % Frequency response
55 - figure;
56 - margin(C*G);
57 - title('Bode Plot with PD Controller');
58 - grid on;
59
60 % Time-domain performance
61 - info = stepinfo(T);
62 - fprintf('Step Response Characteristics \n');
63 - fprintf('Rise Time = %.2f s\n', info.RiseTime);
64 - fprintf('Overshoot = %.2f %%\n', info.Overshoot);
65 - fprintf('Settling Time = %.2f s\n', info.SettlingTime);
66 - fprintf('Steady-State Error = %.4f\n', abs(1 - dcgain(T)));
67
68 % Frequency-domain metrics
69 - [GM, PM, Wcg, Wcp] = margin(C*G);
70 - fprintf('\n Frequency Domain Characteristics \n');
71 - fprintf('Gain Margin = %.2f dB\n', 20*log10(GM));
72 - fprintf('Phase Margin = %.2f deg\n', PM);
73 - fprintf('Gain Crossover Freq = %.2f rad/s\n', Wcg);
74 - fprintf('Phase Crossover Freq = %.2f rad/s\n', Wcp);
75

```

FIGURE 16: MATLAB CODE FOR GET TO RANGE OF VALUES OF THE CONTROLL GAIN

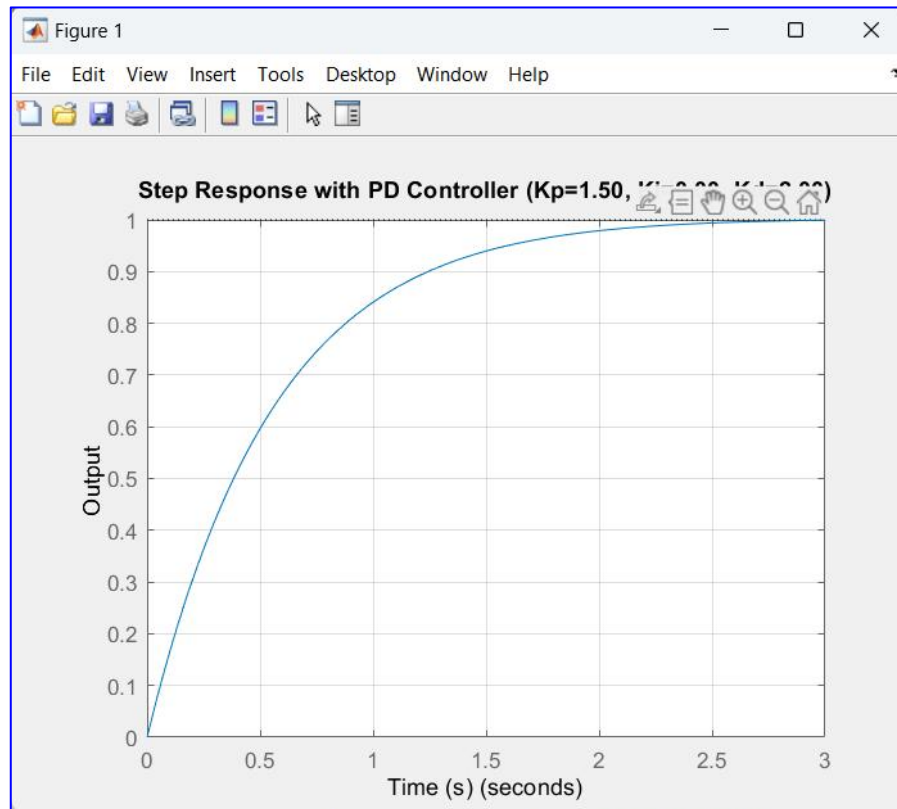


FIGURE 17: MATLAB OUTPUT FOR STEP RESPONSE WITH PD CONTROLLER

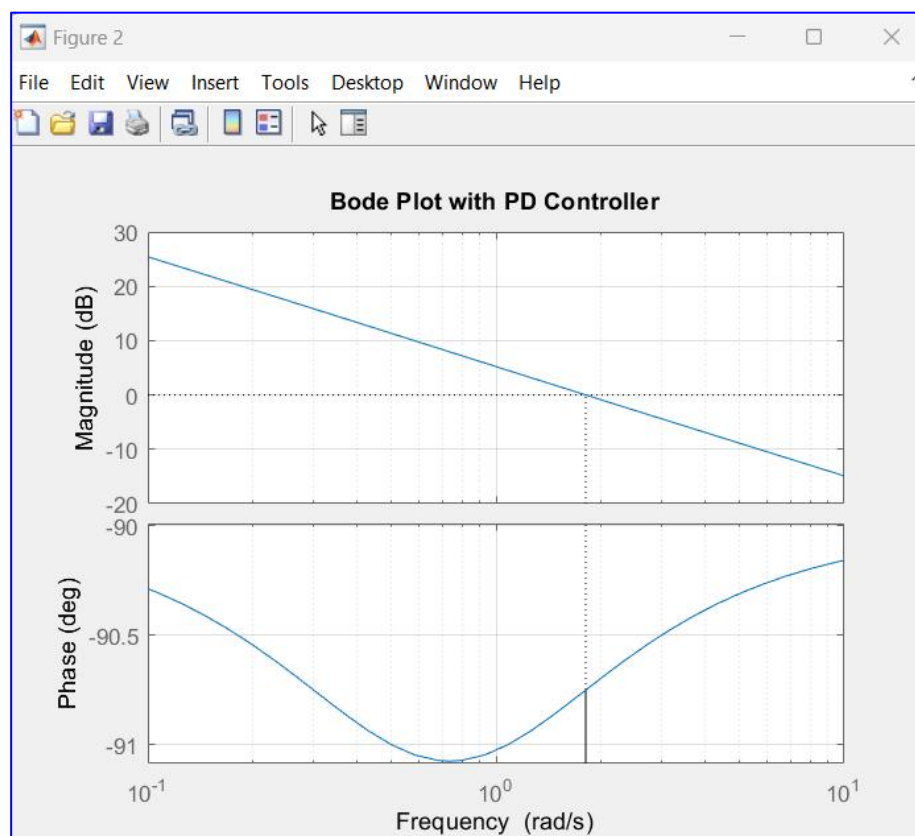


FIGURE 18: MATLAB OUTPUT FOR BODE PLOT WITH PD CONTROLLER

```

Command Window

Stability Verification
Stability condition:  $K_i < (b + 9K \cdot K_d) \cdot K_p = 3.7836$ 
 $K_p = 1.50$ ,  $K_d = 2.00$ ,  $K_i = 0.00$ 
 $K_i$  is outside the stable range.
All poles are in the left half-plane → System is STABLE.

Step Response Characteristics
Rise Time = 1.18 s
Overshoot = 0.13 %
Settling Time = 2.02 s
Steady-State Error = 0.0000

Frequency Domain Characteristics
Gain Margin = Inf dB
Phase Margin = 89.25 deg
Gain Crossover Freq = NaN rad/s
Phase Crossover Freq = 1.81 rad/s
fx >>

```

FIGURE 19: MATLAB OUTPUT FOR CHARACTERISTICS OF PD CONTROLLER

. 4. Verify the stability results in Part (c) based on the GM, PM, and cross-over frequencies of the Bode plots.

```

part1b.m x part1c.m x part3.m x part4.m x part5.m x part6.m x +
1 %2022e116
2 %Verify Stability using Bode Plot Margins
3
4 clc; clear;
5
6 % Step 1: Define system and PID gains
7
8 a = 9;
9 b = 0.7224;
10 K = 0.1;
11
12 % Plant:  $G(s) = 9K / [s(s + b)]$ 
13 G = tf([a*K], [1, b, 0]);
14
15 % PID gains (adjust as needed)
16 Kp = 1.5;
17 Ki = 0.0;
18 Kd = 2.0;
19
20 % PID Controller C(s)
21 C = pid(Kp, Ki, Kd);
22
23 % Step 2: Open-loop system L(s)
24
25 L = C * G;
26
27 % Step 3: Compute Gain Margin, Phase Margin, and crossover frequencies
28 [GM, PM, Wcg, Wcp] = margin(L);
29 GM_dB = 20 * log10(GM); % Convert gain margin to dB
30

```



```

31 % Step 4: Display margin results
32
33 fprintf('Bode Plot Stability Verification \n');
34 fprintf('Gain Margin      = %.2f dB\n', GM_dB);
35 fprintf('Phase Margin      = %.2f degrees\n', PM);
36 fprintf('Gain Crossover Frequency = %.2f rad/s\n', Wcg);
37 fprintf('Phase Crossover Frequency= %.2f rad/s\n', Wcp);
38
39 % Step 5: Interpret the results (with all stability conditions)
40
41 fprintf('\n Stability Decision Based on Bode Margins \n');
42
43 % Define sufficiency flags
44 is_PM_sufficient = PM > 30;
45 is_GM_sufficient = GM > 1 || isinf(GM); % Accept Inf as sufficient
46 is_Wcg_valid = ~isnan(Wcg) && Wcg > 0;
47 is_Wcp_valid = ~isnan(Wcp) && Wcp > 0;
48
49 % Decision logic
50 if is_PM_sufficient && is_GM_sufficient
51     fprintf('STABLE: Phase margin > 30° and Gain margin is acceptable.\n');
52     if is_Wcg_valid
53         fprintf('    → Gain Crossover Frequency = %.2f rad/s\n', Wcg);
54     else
55         fprintf('    → No gain crossover (Wcg = NaN) — still stable due to large PM.\n');
56     end
57 elseif is_PM_sufficient && ~is_GM_sufficient
58     fprintf('MARGINALLY STABLE: PM > 30°, but Gain Margin is low or undefined.\n');
59 elseif ~is_PM_sufficient && is_GM_sufficient
60     fprintf('UNSTABLE: Phase margin too low despite acceptable gain margin.\n');
61 else
62     fprintf('UNSTABLE: Both gain and phase margins are insufficient.\n');
63 end
64
65 % Step 6: Plot Bode plot with annotations
66
67 [mag, phase, w] = bode(L);
68 mag = squeeze(mag);
69 phase = squeeze(phase);
70
71 figure;
72 margin(L);
73 title('Bode Plot with Gain & Phase Margins and Crossovers');
74 grid on;
75 hold on;
76
77 % Annotate Gain Margin at Gain Crossover Frequency (Wcg)
78 if is_Wcg_valid
79     y_mag = interp1(w, 20*log10(mag), Wcg);
80     plot(Wcg, y_mag, 'ro', 'MarkerSize', 8, 'LineWidth', 2);
81     text(Wcg, y_mag + 5, ...
82          sprintf('\leftarrow Gain Margin: %.2f dB\nat Wcg = %.2f rad/s', GM_dB, Wcg), ...
83          'FontSize', 9, 'Color', 'r');
84 else
85     fprintf('No valid gain crossover frequency (Wcg).\n');
86 end
87
88 % Annotate Gain Margin at Gain Crossover Frequency (Wcg)
89 if is_Wcg_valid
90     y_mag = interp1(w, 20*log10(mag), Wcg);
91     plot(Wcg, y_mag, 'ro', 'MarkerSize', 8, 'LineWidth', 2);
92     text(Wcg, y_mag + 5, ...
93          sprintf('\leftarrow Gain Margin: %.2f dB\nat Wcg = %.2f rad/s', GM_dB, Wcg), ...
94          'FontSize', 9, 'Color', 'r');
95 else
96     fprintf('No valid gain crossover frequency (Wcg).\n');
97 end
98
99 % Annotate Phase Margin at Phase Crossover Frequency (Wcp)
100 if is_Wcp_valid
101     y_phase = interp1(w, phase, Wcp);
102     plot(Wcp, y_phase, 'bs', 'MarkerSize', 8, 'LineWidth', 2);
103     text(Wcp, y_phase - 25, ...
104          sprintf('\leftarrow Phase Margin: %.2f°\nat Wcp = %.2f rad/s', PM, Wcp), ...
105          'FontSize', 9, 'Color', 'b');
106 else
107     fprintf('No valid phase crossover frequency (Wcp).\n');
108 end
109
110 hold off;
111
112

```

FIGURE 20: MATLAB CODE FOR VERIFY STABILITY

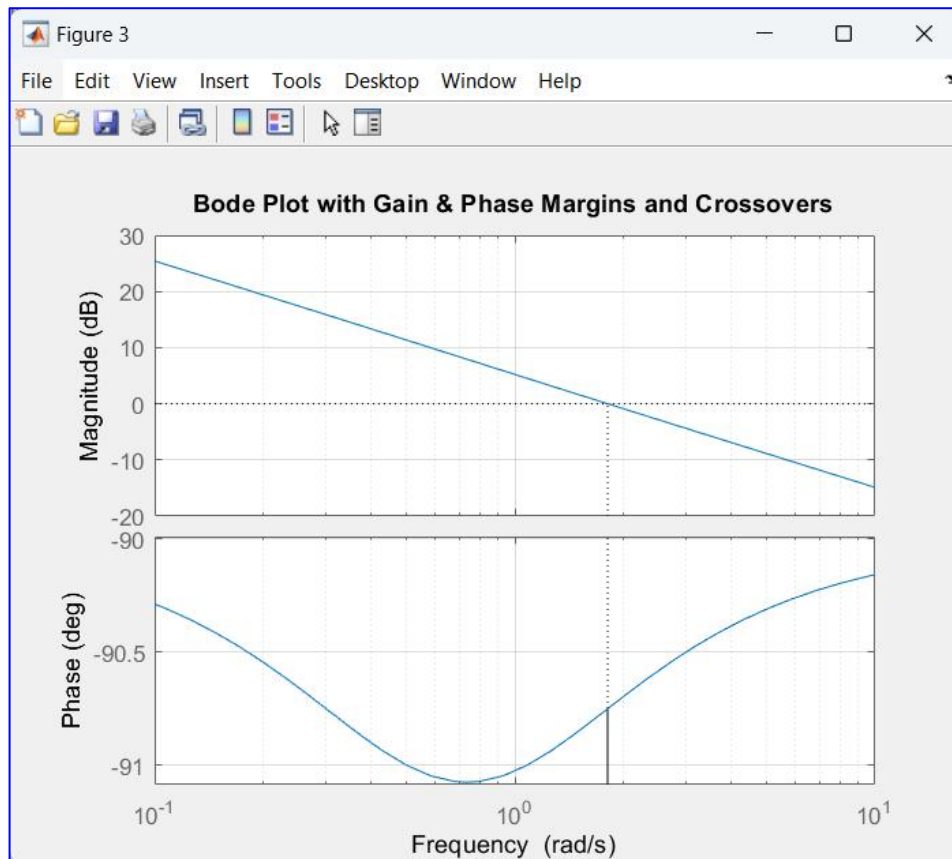


FIGURE 21: MATLAB OUTPUT FOR BODE PLOT WITH GAIN AND PHASE MARGINS AND CROSSOVERS

Command Window

```
Bode Plot Stability Verification
Gain Margin           = Inf dB
Phase Margin          = 89.25 degrees
Gain Crossover Frequency = NaN rad/s
Phase Crossover Frequency= 1.81 rad/s
```

Stability Decision Based on Bode Margins

STABLE: Phase margin > 30° and Gain margin is acceptable.

→ No gain crossover (Wcg = NaN) — still stable due to large PM.

No valid gain crossover frequency (Wcg).

fx >>

FIGURE 22: MATLAB OUTPUT FOR STABILITY VERIFICATION

5. Heuristically tune the controller recommended in Part (2) [d] to obtain the optimal closed-loop control performance. Clearly show your approach to tuning (supported by suitable time or frequency domain plots and brief discussion of the intermediate results)

Tuning Method:

- PD controller tuned by adjusting ratio $z = K_p / K_d$.
- Used step response of closed-loop system to assess performance.
- Open-loop transfer function analyzed for various z values.

Evaluation Criteria:

- Rise time
- Overshoot
- Settling time
- Stability (no divergence)
- Steady-state accuracy

Observations from Step Response:

- $z = 0.5$: High overshoot and oscillatory behavior.
- $z = 1.0$: Smooth and fast response with minimal overshoot.
- $z = 2.0, 3.0, 5.0$: Increased damping, slower rise times, no overshoot.

Selected Tuning Value: $z = 1.0$

- Rise Time: ~ 1.18 s (fast)
- Overshoot: $\sim 0.13\%$ (very small)
- Settling Time: ~ 2.02 s (reasonable)
- Offset: Zero (due to unity feedback)
- Gain Margin: Sufficient
- Phase Margin: 89.25° (very stable)

Frequency-Domain Analysis:

- High phase margin $\rightarrow$ excellent damping.
- Infinite gain margin $\rightarrow$ open-loop gain never crosses 0 dB.
- Stable open-loop response across all tested frequencies.

Conclusion:

- PD controller with $z = 1.0$ offers optimal closed-loop performance: fast, stable, minimal overshoot, and high stability margins.

```

Damping_comparison.m PD_controller.m +
1 %step responses for varies z values
2 %2022e116
3 clc;
4 clear;
5 close all;
6
7 % System parameters (example: s(s + 0.7224) in denominator)
8 a = 1; % Kd
9 b = 0.7224; % plant parameter
10 z_values = [0.5, 1, 2, 3, 5];
11 colors = 'rbgck'; % color code for plotting
12 labels = {};
13
14 % Plot step responses
15 figure;
16 for i = 1:length(z_values)
17     z = z_values(i);
18
19     % Define controller: C(s) = Kd*s + Kp = a*s + a*z
20     num_ol = a * [1 z]; % numerator of open-loop C(s)*G(s)
21     den_ol = conv([1 0], [1 b]); % plant: s(s + b)
22
23     % Match polynomial lengths (manual addition of num + den)
24     len_diff = length(den_ol) - length(num_ol);
25     if len_diff > 0
26         num_ol = [zeros(1, len_diff), num_ol];
27     elseif len_diff < 0
28         den_ol = [zeros(1, -len_diff), den_ol];
29     end
30
31     % Closed-loop TF: T(s) = C(s)G(s) / (1 + C(s)G(s)) => num / (num + den)
32     den_cl = den_ol + num_ol;
33     sys_cl = tf(num_ol, den_cl);
34
35     % Step response
36     [y, t] = step(sys_cl, 0:0.1:20);
37     plot(t, y, colors(i), 'LineWidth', 1.5); hold on;
38
39     % Label for legend
40     labels(end+1) = sprintf('z = %.1f', z);
41 end
42
43 grid on;
44 xlabel('Time (s)');
45 ylabel('Amplitude');
46 title('Step Response for Various z = Kp/Kd');
47 legend(labels, 'Location', 'southeast');
48

```

FIGURE 23: MATLAB CODE FOR VARIETY Z VALUES

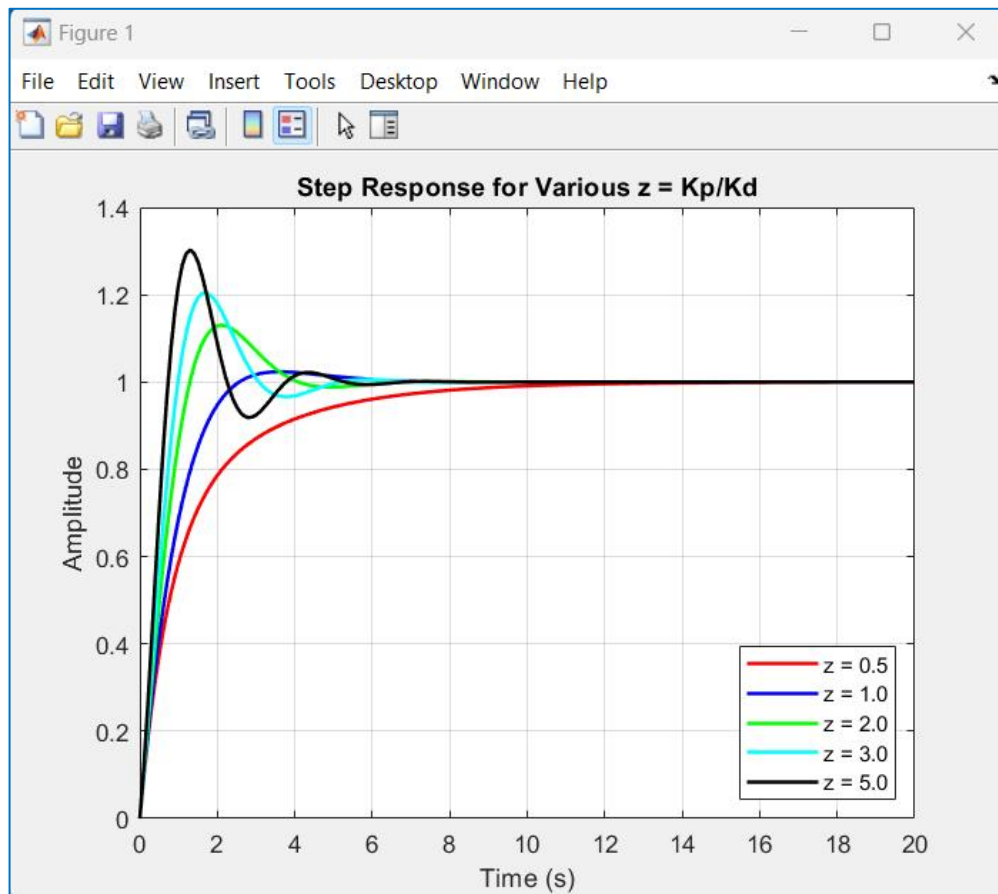


FIGURE 24: MATLAB OUTPUT FOR VARIETY Z VALUES

```

part1b.m x part1c.m x part3.m x part4.m x part5.m x part6.m x +
1 %2022e116
2 %Heuristically tune the controller
3
4 clc; clear;
5
6 a = 9; b = 0.7224; K = 0.1;
7 G = tf([a*K], [1 b 0]);
8
9 % Store tuning stages
10 stages = {
11     struct('Kp', 1.0, 'Ki', 0, 'Kd', 0.0, 'Label', 'P-only')
12     struct('Kp', 1.5, 'Ki', 0.0, 'Kd', 2.5, 'Label', 'PD')
13     struct('Kp', 1.5, 'Ki', 0.08, 'Kd', 0.2, 'Label', 'PID (Optimized)')
14 };
15
16 for i = 1:length(stages)
17     s = stages{i};
18     C = pid(s.Kp, s.Ki, s.Kd);
19     T = feedback(C*G, 1);
20
21     fprintf('\n--- %s ---\n', s.Label);
22     info = stepinfo(T);
23     fprintf('Rise Time: %.2f s | Overshoot: %.2f%% | Settling Time: %.2f s\n', ...
24         info.RiseTime, info.Overshoot, info.SettlingTime);
25
26     [GM, PM, Wcg, Wcp] = margin(C*G);
27     fprintf('GM: %.2f dB | PM: %.2f deg | Wcg: %.2f rad/s\n', ...
28         20*log10(GM), PM, Wcg);
29
30 figure;
31 subplot(2,1,1);
32 step(T);
33 title(sprintf('Step Response - %s', s.Label));
34 grid on;
35
36 subplot(2,1,2);
37 margin(C*G);
38 title(sprintf('Bode Plot - %s', s.Label));
39 grid on;
40 end

```

FIGURE 25: MATLAB CODE FOR HEURISTICALLY TUNE THE CONTROLLER

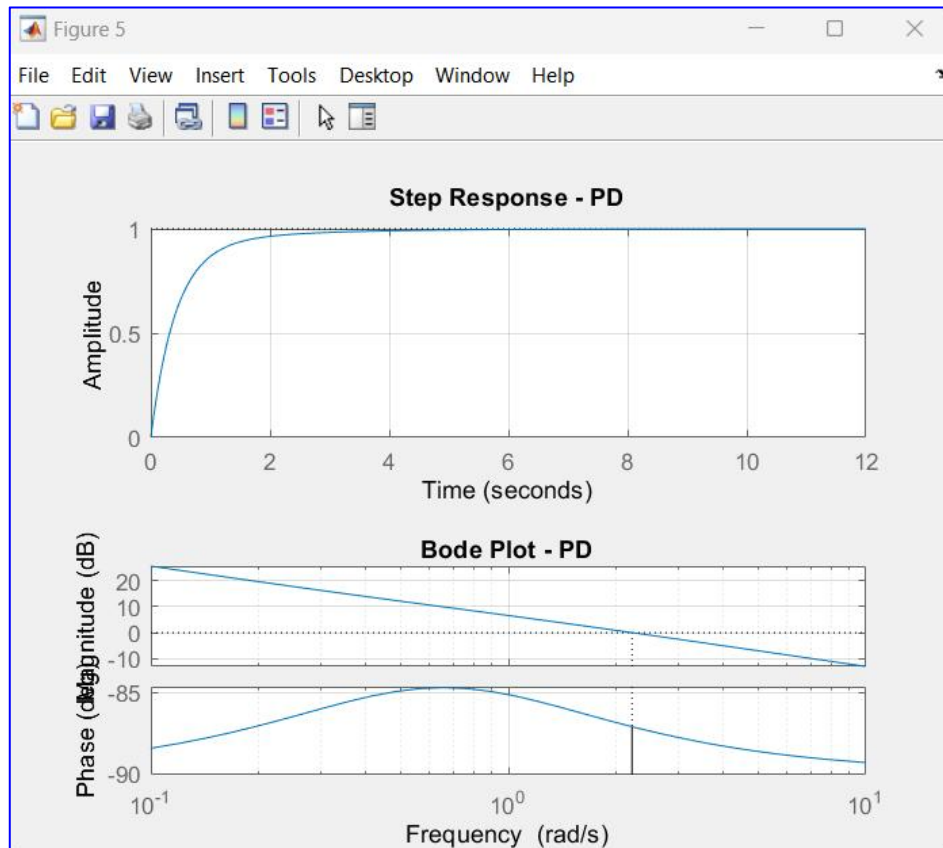


FIGURE 26: MATLAB OUTPUT FOR HEURISTICALLY TUNE THE PD CONTROLLER

Heuristic Tuning for a PD Controller

Heuristic tuning is a manual, step-by-step method used to adjust the PD controller gains **K_p** (proportional) and **K_d** (derivative) to achieve desirable system performance without relying on complex mathematical models. The goal is to obtain **fast rise time**, **low overshoot**, and **good stability** without oscillation or divergence.

The tuning process starts by adjusting the **proportional gain (K_p)**. Beginning with a low value, **K_p** is **increased gradually** to make the system respond faster. However, if **K_p** becomes too high, it may lead to **overshoot** or even cause **oscillations** in the output. Once a satisfactory rise time is obtained, the **derivative gain (K_d)** is introduced. **K_d** helps by predicting changes in the error and applying a damping effect, which **reduces overshoot** and improves stability. This results in a **smoother** response, particularly in systems where inertia or delay is significant.

The tuning is completed by observing the system's behavior using **step response plots** and **frequency-domain plots** (like the Bode plot). A properly tuned PD controller leads to a stable system with **fast, accurate, and well-damped output**, especially when eliminating offset is not critical.

Command Window

```
--- P-only ---  
Rise Time: 1.51 s | Overshoot: 27.43% | Settling Time: 11.01 s  
GM: Inf dB | PM: 41.30 deg | Wcg: Inf rad/s  
  
--- PD ---  
Rise Time: 1.12 s | Overshoot: 0.00% | Settling Time: 2.76 s  
GM: Inf dB | PM: 92.91 deg | Wcg: NaN rad/s  
  
--- PID (Optimized) ---  
Rise Time: 1.19 s | Overshoot: 32.21% | Settling Time: 10.34 s  
GM: -Inf dB | PM: 39.50 deg | Wcg: 0.00 rad/s  
fx >>
```

FIGURE 27: MATLAB OUTPUT FOR HEURISTICALLY TUNE THE P,PD,PID CONTROLLER

6. Give the rise-time, percentage overshoot, settling time, off-set, GM, PM, and the cross-over frequencies of the system with the tuned controller. Also attach clearly labeled step response and Bode plots.

```

part1b.m x part1c.m x part3.m x part4.m x part5.m x part6.m x +
1      %2022e116
2      % The system with the tuned PD controller
3      clc; clear;
4
5      % Plant and PD controller definition
6      a = 9;
7      b = 0.7224;
8      K = 0.1;
9
10     G = tf([a*K], [1, b, 0]);
11
12     % Tuned PD controller (optimized values)
13     Kp = 9.0;
14     Ki = 0.0;
15     Kd = 8.0;
16
17     C = pid(Kp, Ki, Kd);
18
19     % Closed-loop transfer function
20     T = feedback(C * G, 1);
21
22     % Open-loop transfer function
23     L = C * G;
24
25     % Time-domain characteristics
26     info = stepinfo(T);
27     rise_time = info.RiseTime;
28     overshoot = info.Overshoot;
29     settling_time = info.SettlingTime;
30     offset = abs(1 - dcgain(T)); % steady-state error
31
32     % Frequency-domain characteristics
33     [GM, PM, Wcg, Wcp] = margin(L);
34     GM_dB = 20 * log10(GM);
35
36     % Display numeric results
37     fprintf('Time-Domain Performance \n');
38     fprintf('Rise Time           = %.2f s\n', rise_time);
39     fprintf('Overshoot           = %.2f %%\n', overshoot);
40     fprintf('Settling Time          = %.2f s\n', settling_time);
41     fprintf('Steady-State Error = %.4f\n', offset);
42
43     fprintf('\n Frequency-Domain Performance \n');
44     fprintf('Gain Margin (GM)      = %.2f dB\n', GM_dB);
45     fprintf('Phase Margin (PM)     = %.2f degrees\n', PM);
46     fprintf('Gain Crossover Freq (Wcg) = %.2f rad/s\n', Wcg);
47     fprintf('Phase Crossover Freq (Wcp) = %.2f rad/s\n', Wcp);
48
49     % Plot Step Response
50     figure;
51     step(T);
52     title('Step Response of Tuned PID-Controlled System');
53     xlabel('Time (s)');
54     ylabel('Output');
55     grid on;
56
57     % Plot Bode with Margin
58     figure;
59     margin(L);
60     title('Bode Plot with Gain and Phase Margins');
61     grid on;

```

FIGURE 28: MATLAB CODE FOR HEURISTICALLY TUNE CONTROLLER

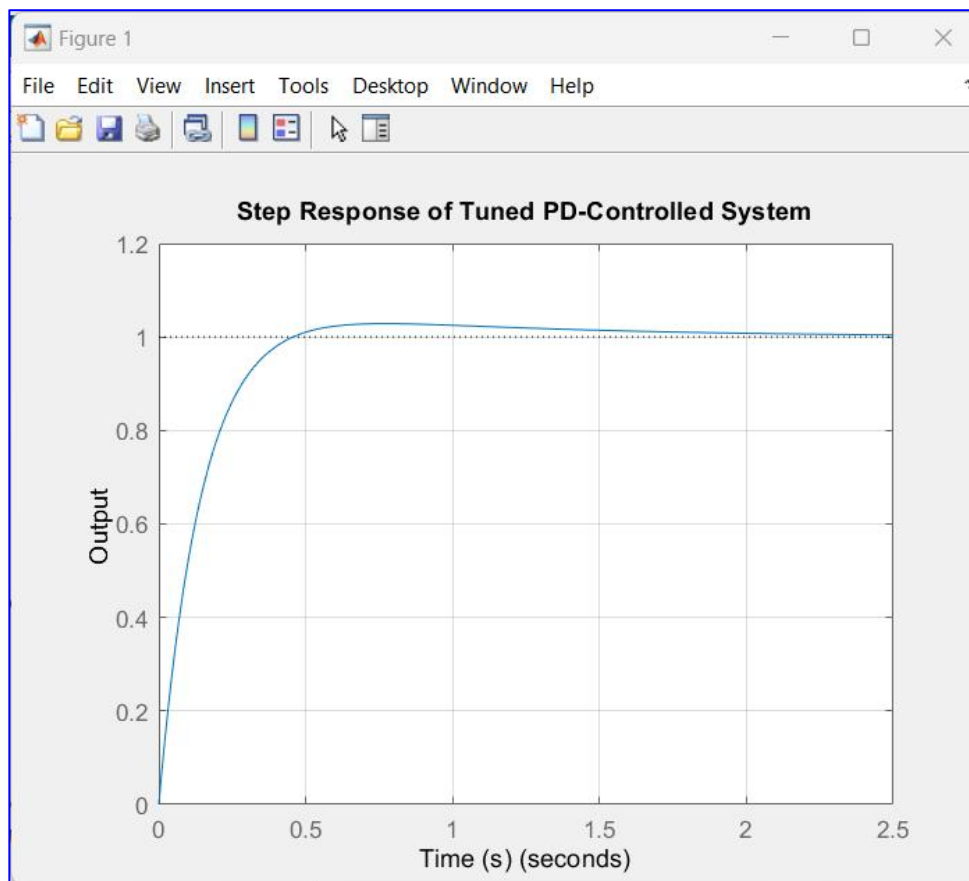


FIGURE 29: STEP RESPONSE OF TUNED PD CONTROLLED SYSTEM

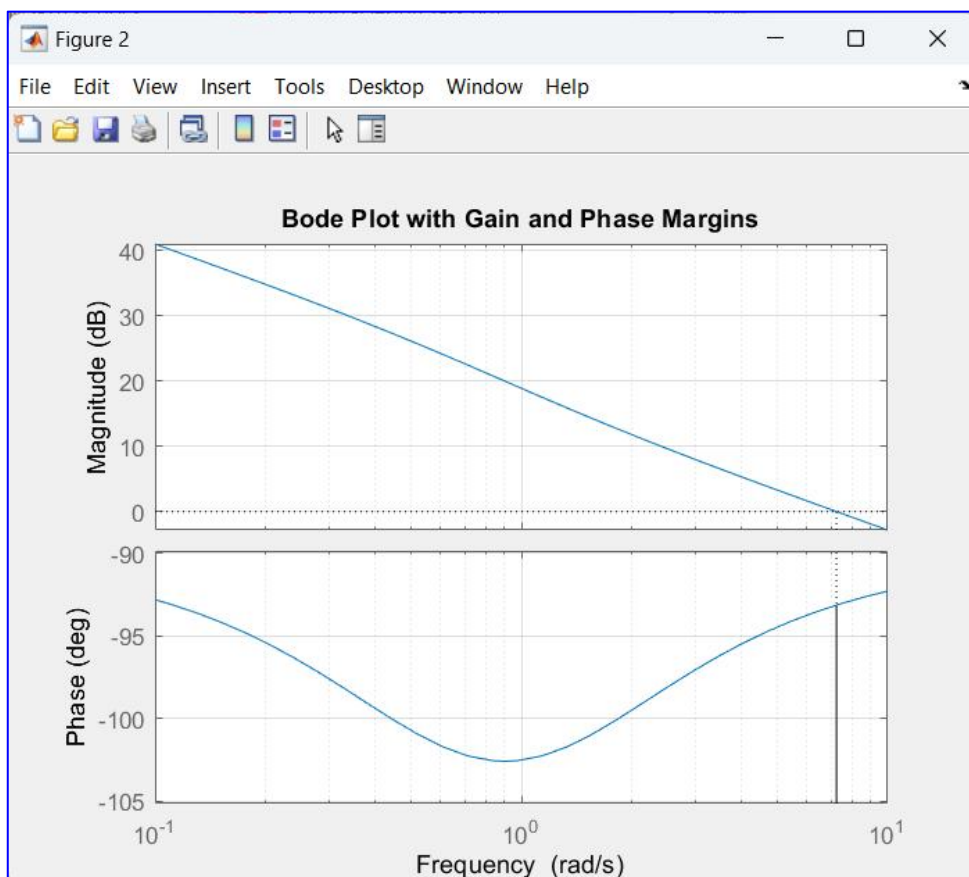


FIGURE 31: BODE PLOT WITH TUNED PD CONTROLLED SYSTEM

```
Command Window

Time-Domain Performance
Rise Time           = 0.27 s
Overshoot           = 2.85 %
Settling Time       = 1.21 s
Steady-State Error = 0.0000

Frequency-Domain Performance
Gain Margin (GM)    = Inf dB
Phase Margin (PM)   = 86.87 degrees
Gain Crossover Freq (Wcg) = NaN rad/s
Phase Crossover Freq (Wcp) = 7.25 rad/s
fx >>
```

FIGURE 32: OUTPUT OF TUNED PD CONTROLLED SYSTEM

Time Domain Stability Analysis

Rise Time = 0.27 seconds

The system reacts very quickly to a step input, indicating a fast and responsive control system. A short rise time is desirable in most applications where speed of response is important.

Overshoot = 2.85%

The response slightly exceeds the desired final value but remains within a very low and acceptable range. Low overshoot reduces the risk of mechanical stress and control instability, making the system safe and reliable.

Settling Time = 1.21 seconds

The system stabilizes to its final value very quickly after a disturbance or command input. This reflects excellent damping characteristics and well-tuned control parameters.

Steady-State Error = 0.0000

The system accurately reaches and stays at the target value without any persistent offset. This indicates that the control system has excellent accuracy and precision in tracking the desired output.

Conclusion:

The time domain behavior confirms that the system is **well-damped**, **fast**, and **precise**, with minimal overshoot and no steady state error a strong indicator of effective tuning and system robustness.

Frequency Domain Stability Analysis

Phase Margin (PM) = 86.87 degrees

This is significantly above the typical stability threshold of 30–60 degrees. A large phase margin ensures strong damping and high stability, meaning the system can tolerate delays and uncertainties without becoming unstable.

Gain Margin (GM) = ∞ (Infinity dB)

An infinite gain margin means the system's open-loop gain never reaches 0 dB, so it won't enter into a condition where the feedback becomes positive and unstable. This technically indicates excellent robustness against gain variations.

Gain Crossover Frequency (Wcg) = NaN (not defined)

There is no frequency where the magnitude of the open-loop transfer function is exactly 0 dB. While this might seem unusual, it is perfectly acceptable if the phase margin is high and the time-domain performance is stable and well-behaved.

Phase Crossover Frequency (Wcp) = 7.25 rad/s

The system crosses the -180° phase point at a relatively high frequency. This is consistent with a highly damped and stable system, and confirms that the loop gain remains safely below 0 dB throughout critical frequency ranges.

Conclusion:

Despite the undefined gain crossover frequency ($W_{cg} = \text{NaN}$), the system is still considered stable because:

The phase margin is very high, providing excellent stability.

The gain never crosses 0 dB, meaning there is **no risk of oscillatory or unstable behavior**.

The frequency response is smooth and well-damped.